Scenario Based Interview Questions

Spark SQL

PySpark



Ganesh. R

SwipeRight

Problem Statement



DataFrame containing employee information across different departments. Each record includes an employee ID (eid), department name (dept), and a score (scores). The goal is to perform the following operations:

1. Identify Top Scorers: For each department, identify the employee(s) with the highest score. If there are multiple employees with the same highest score in a department, they should all be considered top scorers.
2. Right Join with All Employees: Combine the list of all employees (eid, dept) with the top scorers, ensuring that every employee is present in the final result, even if they are not a top scorer in their department. The scores for non-top scorers should be null.
3. Order the Results: The final output should be sorted by the employee ID (eid).

Input Table Data

```
schema = StructType([
  StructField("eid", IntegerType(), True),
  StructField("dept", StringType(), True),
  StructField("scores", FloatType(), True)
])

# step 4: Create DataFrame with the specified data
data = [
  (1, 'd1', 1.0),
  (2, 'd1', 5.28),
  (3, 'd1', 4.0),
  (4, 'd2', 8.0),
  (5, 'd1', 2.5),
  (6, 'd2', 7.0),
  (7, 'd3', 9.0),
  (8, 'd4', 10.2)
]

df = spark.createDataFrame(data, schema)
```

Expected Output

eid	dept	scores
1	d1	5.28
2	d1	5.28
3	d1	5.28
4	d2	8
5	d1	5.28
6	d2	8
7	d3	9
8	d4	10.19

#Problem Statement:

###Employee Department Scores: I am giving a dataset containing employee information across different departments. Each record includes an employee ID (eid), department name (dept), and a score (scores). The goal is to perform the following operations:

###Identify Top Scorers: For each department, identify the employee(s) with the highest score. If there are multiple employees with the same highest score in a department, they should all be considered top scorers.

###Right Join with All Employees: Combine the list of all employees (eid, dept) with the top scorers, ensuring that every employee is present in the final result, even if they are not a top scorer in their department. The scores for non-top scorers should be null.

###Order the Results: The final output should be sorted by the employee ID (eid).

```
# step 1: Import necessary PySpark libraries
from pyspark.sql import SparkSession
from pyspark.sql.types import StructType, StructField, IntegerType,
StringType, FloatType
from pyspark.sql import functions as F
from pyspark.sql.window import Window

# step 2: Initialize Spark session
spark = SparkSession.builder \
    .appName("Create and Insert Table") \
    .getOrCreate()

# step 3: Define schema for the table
schema = StructType([
    StructField("eid", IntegerType(), True),
    StructField("dept", StringType(), True),
    StructField("scores", FloatType(), True)
])

# step 4: Create DataFrame with the specified data
data = [
    (1, 'd1', 1.0),
    (2, 'd1', 5.28),
    (3, 'd1', 4.0),
    (4, 'd2', 8.0),
    (5, 'd1', 2.5),
    (6, 'd2', 7.0),
    (7, 'd3', 9.0),
    (8, 'd4', 10.2)
]

df = spark.createDataFrame(data, schema)

# step 5: Save DataFrame as a table in the default database (or
```

```
specific database)
df.createOrReplaceTempView("empdept_tbl")

# step 6: Display the DataFrame content (optional)
df.display()
```

#Spark SQL

```
%sql
with t1 as(
  select
    *,
    dense_rank() over(
      partition by dept
      order by
        scores desc
    ) dn
  from
    empdept_tbl qualify dn = 1
),
t2 as(
  select
    eid,
    dept
  from
    empdept_tbl
)
select
  t2.eid,
  t2.dept,
  round(t1.scores, 2) as scores
from
  t1
  right join t2 on t1.dept = t2.dept
order by
  1;
```

#PySpark

```
# Step 1: Create window specification for dense_rank
window_spec = Window.partitionBy("dept").orderBy(F.desc("scores"))

# Step 2: Apply dense_rank and filter for rank = 1 (simulate QUALIFY
clause)
df_with_rank = df.withColumn("dn",
F.dense_rank().over(window_spec)).filter(F.col("dn") == 1)

# Step 3: Round the scores to the nearest integer
df_with_rank = df_with_rank.withColumn("scores",
```

```
F.round(F.col("scores"), 1))

# Step 4: Create t1 as the result of the above operation
t1 = df_with_rank.select("dept", "scores")

# Step 5: Create t2 as a selection of eid and dept
t2 = df.select("eid", "dept")

# Step 6: Perform right join between t1 and t2
result_df = t2.join(t1, on="dept", how="right")

# Step 7: Order by eid
result_df = result_df.orderBy("eid")
# display the final result
result_df.display()
```

#Explanation:

###Window Function: We use Window.partitionBy("dept").orderBy(F.desc("scores")) to define the partitioning and ordering for the DENSE_RANK window function.

###Filtering: Since PySpark doesn't support the QUALIFY clause, we filter the result after calculating the rank using .filter(F.col("dn") == 1).

###Join Operation: We then perform a right join between the t1 and t2 DataFrames based on the dept column.

###Ordering: Finally, the result is ordered by eid.

This approach should give you the desired output from your original SQL query but implemented in PySpark.

IF YOU FOUND THIS POST USEFUL, PLEASE SAVE IT.



Ganesh. R

COMMENT

REPOST

THANK YOU

For Your Support



I Appreciate for your support on My Account, I will Never Stop to Share the Knowledge.

rganesh203 (Ganesh R)

rganesh203 (Ganesh R)

rganesh203 (Ganesh R)

rganesh203 (Ganesh R)

rganesh203 (Ganesh R)

rganesh203 (Ganesh R)